



Tsena Malagasy

Application mobile

Marché agricole direct producteur–consommateur pour la coopérative agricole malgache

Réalisé par

RAKOTONIRINA Tinasoa Jerrick — ETU001841

RASOANAIVO Ilo Ny Aina — ETU001881

Projet mobile — Module M1

Septembre 2026

Sommaire

1. Présentation du projet
 - 1.1 Vision du projet
 - 1.2 Problématique et proposition de valeur
 - 1.3 Objectifs clés
 - 1.4 Répartition des tâches
2. Réalisation de l'application
 - 2.1 Analyse de l'existant
 - 2.2 Conception de l'application
 - 2.2.1 Modèle de données
 - 2.2.2 Choix technologiques
 - 2.2.3 Architecture
 - 2.3 Fonctionnalités
 - 2.3.1 Écran Liste des annonces
 - 2.3.2 Écran Détail d'une annonce
 - 2.3.3 Écran Publier une annonce
 - 2.3.4 Écran Mes publications
 - 2.4 Limites de la version actuelle
 - 2.5 Confidentialité des données
 - 2.6 Perspectives d'évolution
3. Annexe technique
 - 3.1 Null safety et data class
 - 3.2 Coroutines
 - 3.3 Cycle de vie et Intents
 - 3.4 Jetpack Compose (état)
 - 3.5 Navigation (NavHost et routes)
 - 3.6 Architecture MVVM (ViewModel + StateFlow)
 - 3.7 Room

1. Présentation du projet

1.1 Vision du projet

L'application mobile Tsena Malagasy ambitionne de devenir l'outil de référence pour connecter directement les petits producteurs agricoles malgaches (vanille, café, girofle, riz, litchi...) aux acheteurs locaux, sans intermédiaire. Conçue pour répondre aux problèmes d'accès à l'information sur les prix et la disponibilité des produits, elle centralise les annonces de vente par village et par produit.

1.2 Problématique et proposition de valeur

À Madagascar, un producteur vend souvent via plusieurs intermédiaires avant d'atteindre le consommateur final : chaque intermédiaire prélève une marge, et le producteur touche une fraction du prix réel du marché. Tsena Malagasy apporte une réponse concrète :

- Mise en relation directe : le producteur publie une annonce (produit, quantité, prix ou « à négocier », village, contact) ; l'acheteur la consulte et appelle directement.
- Persistance locale (offline-first) : les annonces déjà consultées restent disponibles même sans connexion, une contrainte réaliste en zone rurale.
- Simplicité d'usage : filtrage par village et tri par prix, pensés pour un usage rapide sur un téléphone d'entrée de gamme.

1.3 Objectifs clés

- Réduire le nombre d'intermédiaires entre le producteur et l'acheteur.
- Donner au producteur une meilleure visibilité sur les prix pratiqués.
- Offrir une interface simple, développée avec Jetpack Compose, utilisable sans connexion permanente.

1.4 Répartition des tâches

Membre	Contributions principales
RAKOTONIRINA Tinasoa Jerrick	Couche données (Room : entité, DAO, base) — Donnees.kt ; couche état (ViewModel, StateFlow, combine) — AnnonceViewModel.kt ; rédaction du présent rapport.
RASOANAIVO Ilo Ny Aina	Écrans Jetpack Compose (Liste, Détail, Publier) et navigation — MainActivity.kt ; tests manuels sur émulateur et prise des captures d'écran.

2. Réalisation de l'application

2.1 Analyse de l'existant

Actuellement, un producteur qui veut vendre sa récolte dépend du bouche-à-oreille ou d'un collecteur qui fixe son propre prix. L'acheteur, lui, n'a aucune visibilité sur ce qui est disponible dans les villages environnants sans s'y déplacer. Cette absence d'information centralisée profite aux intermédiaires plutôt qu'aux deux parties directement concernées.

2.2 Conception de l'application

2.2.1 Modèle de données

Le modèle métier repose sur une seule entité, volontairement simple pour un premier projet : l'annonce publiée par un producteur.

Champ	Type	Rôle
id	Int	Identifiant unique, généré automatiquement
nomProduit	String	Nom du produit (ex. Vanille Bourbon)
village	String	Village d'origine, utilisé pour le filtrage
quantiteKg	Double	Quantité disponible en kilogrammes
prixKg	Double ?	Prix au kilo ; nul si le producteur laisse « à négocier »
nomProducteur / telephone	String	Coordonnées affichées sur l'écran de détail
dateISO	String	Date de publication ; utilisée pour le tri « Récentes » et l'affichage « Publiée le... »

vendue	Boolean	Marque une annonce comme conclue sans la supprimer
estMienne	Boolean	true si l'annonce a été publiée depuis cet appareil (affichée dans l'onglet Mes publications) ; false pour les annonces du jeu de démonstration

2.2.2 Choix technologiques

Pour le développement de l'application mobile, le langage Kotlin a été associé à Jetpack Compose pour la construction de l'interface, avec Room pour la persistance locale — la même stack que celle utilisée dans les mini-TP du module.

2.2.3 Architecture

L'application suit une architecture en trois couches :

Couche	Rôle	Fichier
UI (Compose)	Affiche l'état, remonte les événements (clics) vers le ViewModel	MainActivity.kt
État (ViewModel)	Porte l'état de l'écran dans un StateFlow ; survit à la rotation et à la navigation	AnnonceViewModel.kt
Données (Room)	Source de vérité : entité, DAO, base de données	Donnees.kt

Le flux est unidirectionnel : l'état descend de la base de données vers l'écran via des Flow ; les événements utilisateur remontent vers le ViewModel via des fonctions (publier, changerVillage, marquerVendue).

L'application affiche désormais une barre de navigation basse (NavigationBar) à deux onglets — Accueil et Mes publications — tous deux rattachés au même AnnonceViewModel partagé. La fonction BarreNavigation(navController) observe la route active via currentBackStackEntryAsState() pour surligner l'onglet sélectionné (voir Figure 14 en annexe).

2.3 Fonctionnalités

2.3.1 Écran Liste des annonces

L'écran d'accueil affiche l'ensemble des annonces actives. Il se compose de trois éléments :

- Puces de tri : Récentes / Prix croissant.
- Puces de filtre : par village, générées dynamiquement à partir des annonces existantes.
- Liste : une carte par annonce, avec produit, quantité, village et prix (ou « à négocier »).

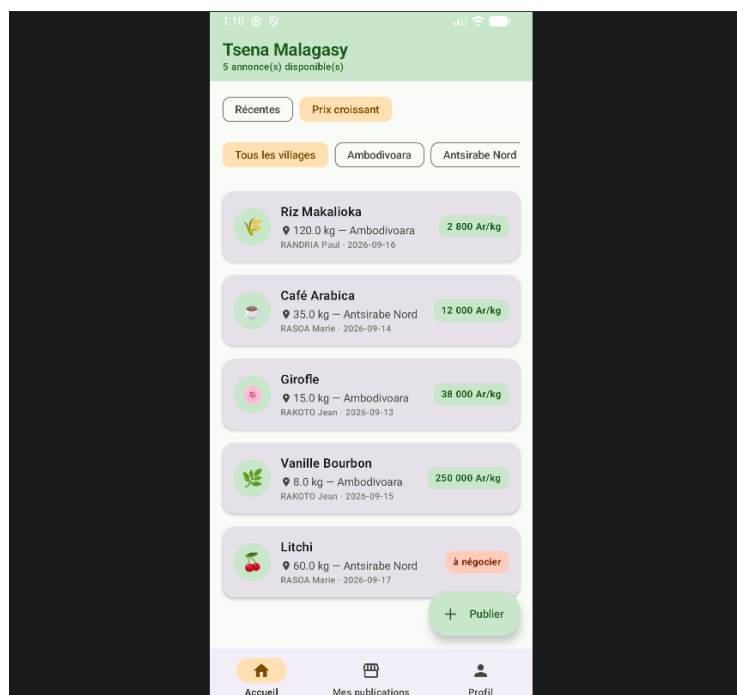


Figure 1 – Écran « Liste des annonces »

2.3.2 Écran Détail d'une annonce

Cet écran présente le détail d'une annonce et permet de contacter directement le producteur par téléphone. Un second bouton, Marquer comme vendue, permet de retirer l'annonce de la liste active (vendue = true) sans la supprimer de la base, avec confirmation par un message Toast.

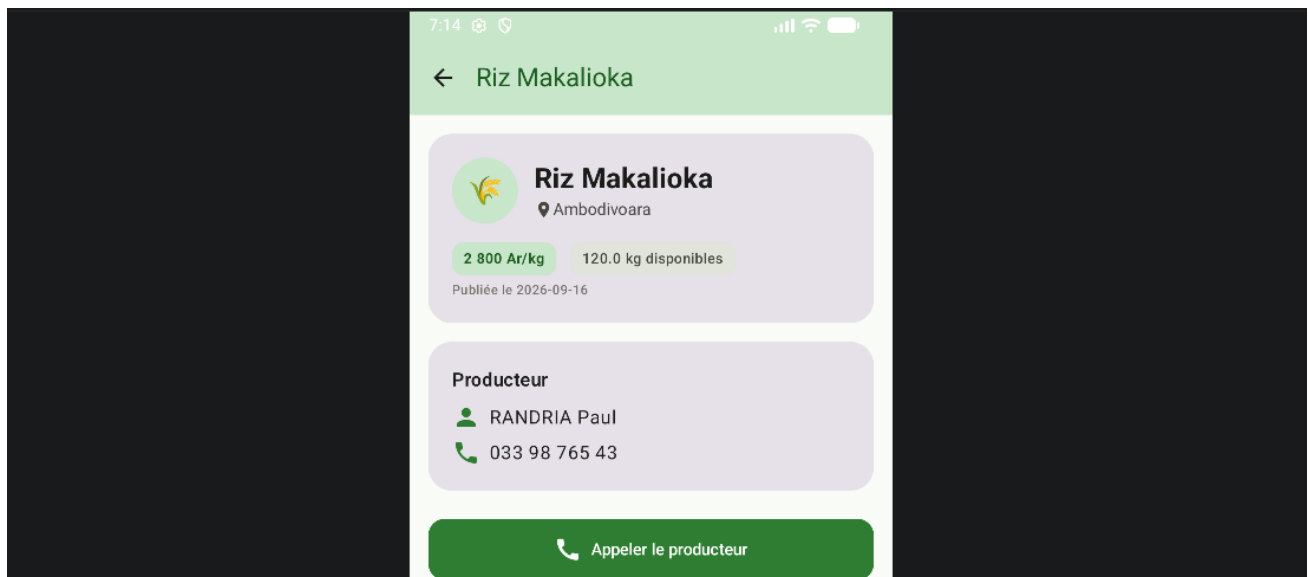


Figure 2 – Écran « Détail d'une annonce »

2.3.3 Écran Publier une annonce

Ce formulaire permet à un producteur de publier une nouvelle annonce. Le champ prix peut être laissé vide : l'annonce apparaît alors comme « à négocier ».

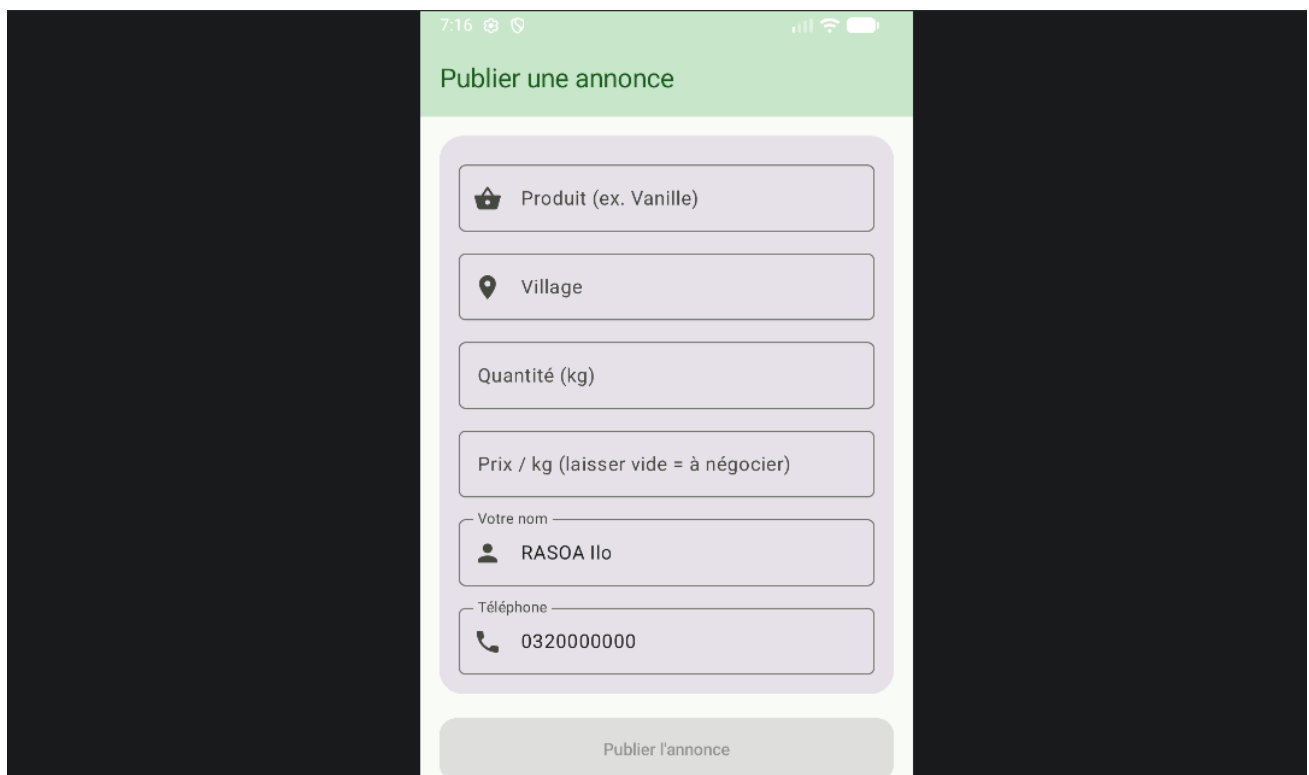


Figure 3 – Écran « Publier une annonce »

2.3.4 Écran Mes publications (nouveau)

Cet onglet liste uniquement les annonces publiées par l'utilisateur depuis cet appareil (`estMienne = true`), vendues incluses. Chaque carte affiche un contrôle de stock – / quantité / + qui appelle `viewModel.modifierStock()` pour ajuster `quantiteKg` en base (Room), désactivé si l'annonce est marquée vendue. Un état vide invite à publier une première annonce quand la liste est vide.

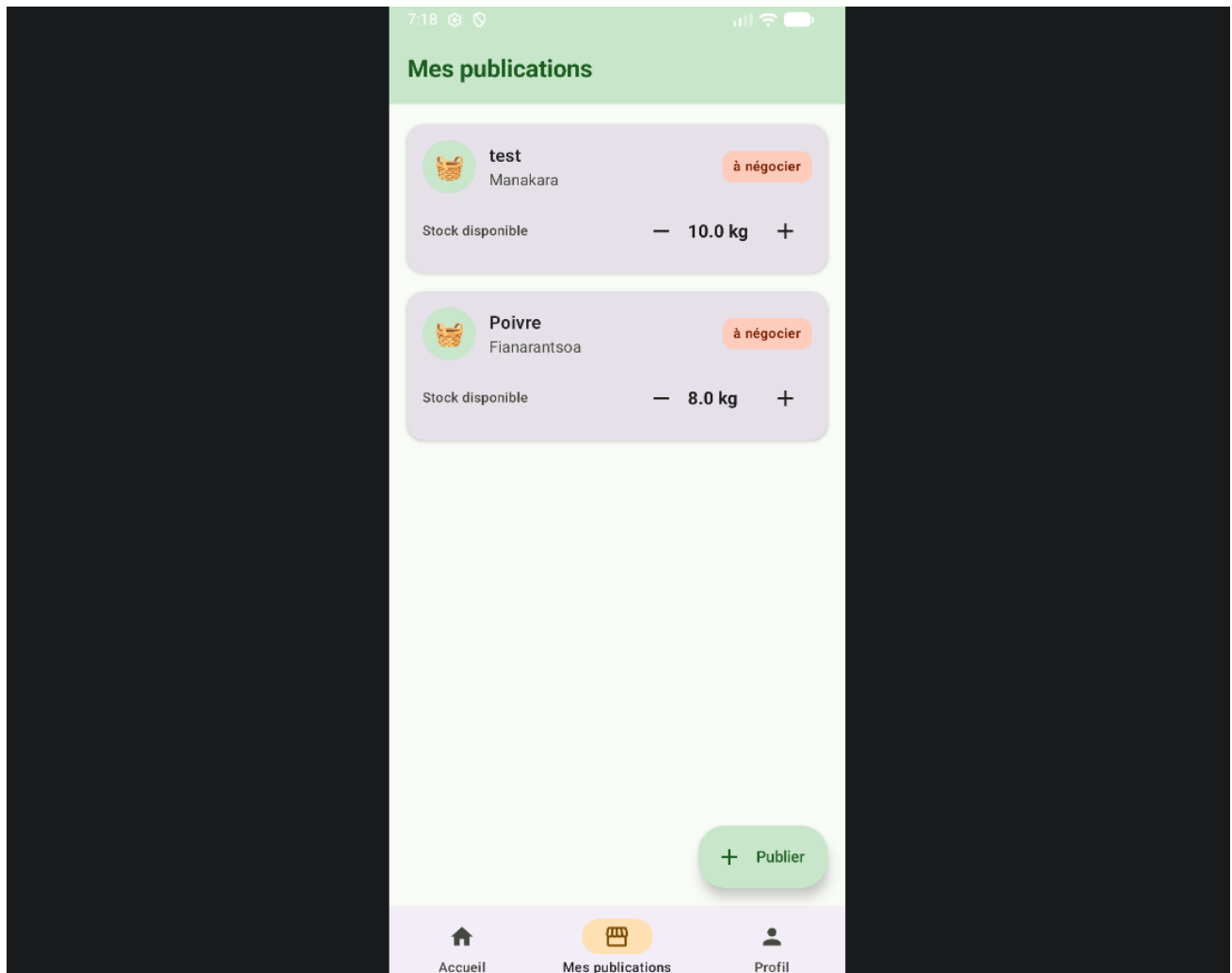


Figure 3bis – Écran « Mes publications »

2.4 Limites de la version actuelle

Par souci de transparence, voici ce que cette première version ne couvre pas encore :

- Aucun compte utilisateur : n'importe qui peut publier une annonce sans identification préalable. Le champ `estMienne` distingue désormais localement « mes annonces » des autres sur un même appareil, mais sans compte ni identité vérifiée — une réinstallation ou un autre appareil ne reconnaît pas les mêmes annonces comme siennes.
- Aucun paiement intégré : la transaction reste hors application, en liquide, comme aujourd'hui.
- Aucune synchronisation serveur : la base Room est locale à chaque installation ; deux téléphones ne voient pas les mêmes annonces.
- Aucun contrôle de véracité : rien n'empêche de publier une annonce fictive ou un prix erroné.

2.5 Confidentialité des données

L'écran de détail affiche en clair le nom et le numéro de téléphone du producteur, sans que celui-ci ne contrôle qui consulte cette information — n'importe quel utilisateur de l'application peut y accéder en un clic. Ce compromis est acceptable pour un prototype pédagogique, mais devrait être reconsidéré avant tout déploiement réel : une messagerie interne à l'application, ou un numéro masqué révélé seulement après une demande de contact explicite, réduirait l'exposition des données personnelles du producteur.s

2.6 Perspectives d'évolution

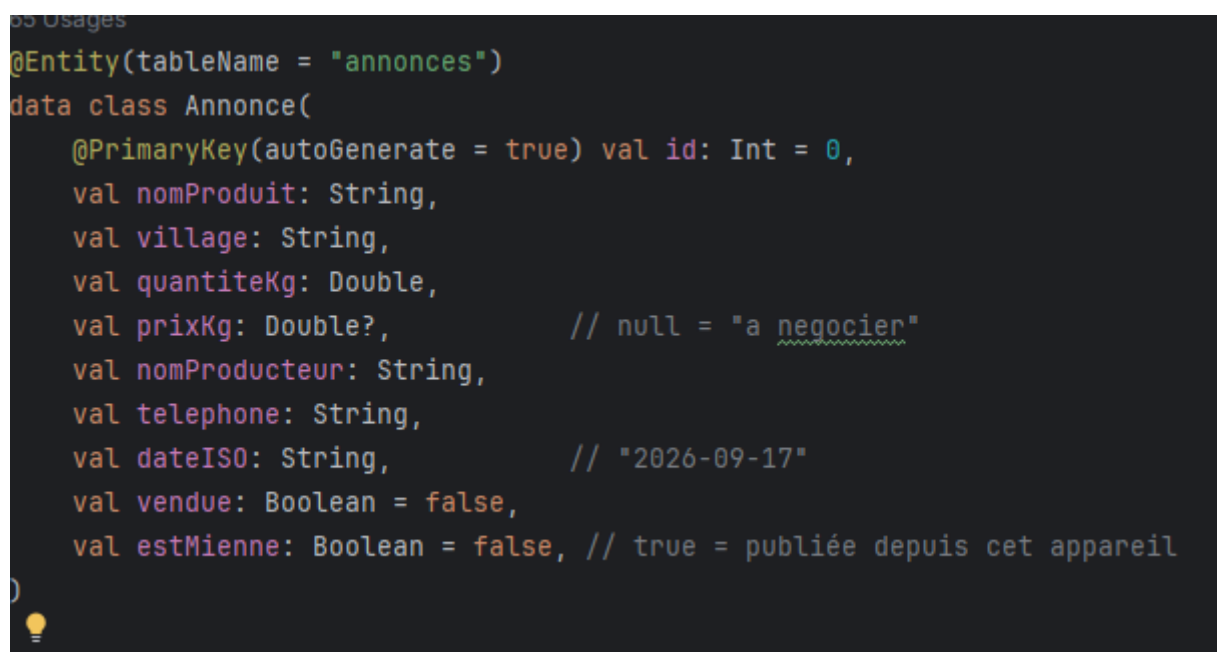
- Backend et synchronisation offline-first (patron démontré en cours, projet demosync) pour partager les annonces entre appareils.
- Messagerie in-app pour contacter le producteur sans exposer directement son numéro de téléphone.
- Géolocalisation pour trier les annonces par distance plutôt que par village saisi manuellement.
- Historique des prix par produit, pour aider le producteur à ne pas se faire sous-payer.

3. Annexe technique

Cette annexe illustre, par des captures du code source dans Android Studio, la mise en œuvre des principales notions vues dans les mini-TP du module.

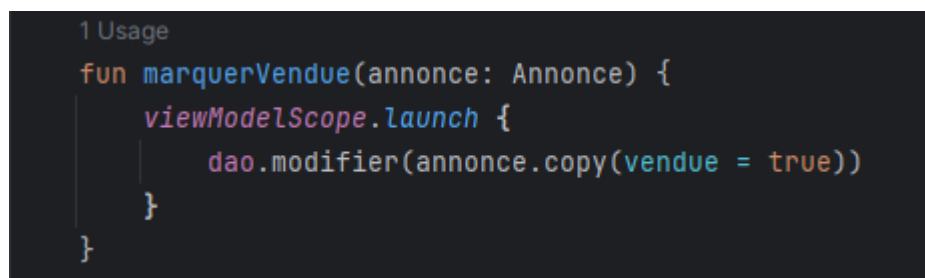
3.1 Null safety et data class

Utilisation de Double? (prix nullable), ?.let et ?: pour représenter une annonce « à négocier » sans opérateur !!, et de copy() pour marquer une annonce comme vendue sans muter l'originale.



```
35 Usages
@Entity(tableName = "annonces")
data class Annonce(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val nomProduit: String,
    val village: String,
    val quantiteKg: Double,
    val prixKg: Double?,           // null = "à négocier"
    val nomProducteur: String,
    val telephone: String,
    val dateISO: String,           // "2026-09-17"
    val vendue: Boolean = false,
    val estMienne: Boolean = false, // true = publiée depuis cet appareil
)
```

Figure 4 – L'entité Annonce — champ prixKg nullable



```
1 Usage
fun marquerVendue(annonce: Annonce) {
    viewModelScope.launch {
        dao.modifier(annonce.copy(vendue = true))
    }
}
```

Figure 5 – copy() pour marquer une annonce vendue sans la muter

3.2 Coroutines

Utilisation de `viewModelScope.launch` et de fonctions `suspend` dans le DAO et le ViewModel, pour des écritures en base non bloquantes.

```
init {  
    // Premier lancement  
    viewModelScope.launch {  
        annoncesInitiales.forEach { dao.inserer( annonce = it) }  
    }  
}
```

Figure 6 – Insertion du jeu de données initial au premier lancement

```
/** Publier une nouvelle annonce */  
1 Usage  
fun publier(  
    nomProduit: String,  
    village: String,  
    quantiteKg: Double,  
    prixKg: Double?,  
    nomProducteur: String,  
    telephone: String,  
    dateISO: String,  
) {  
    viewModelScope.launch {  
        dao.inserer(  
            Annonce(  
                nomProduit = nomProduit,  
                village = village,  
                quantiteKg = quantiteKg,  
                prixKg = prixKg,  
                nomProducteur = nomProducteur,  
                telephone = telephone,  
                dateISO = dateISO,  
                estMienne = true,  
            )  
        )  
    }  
}
```

Figure 7 – Fonction `publier()` — écriture suspend depuis le formulaire

3.3 Cycle de vie et Intents

Utilisation d'un Intent implicite (ACTION_DIAL) pour déléguer l'appel téléphonique à l'application Téléphone du système, sans écrire de logique d'appel soi-même.

```
Button(  
    onClick = {  
        // Intent implicite : appeler le producteur  
        val intent = Intent( action = Intent.ACTION_DIAL, uri = "tel:${a.telephone}".toUri())  
        context.startActivity( p0 = intent)  
    },  
    modifier = Modifier.fillMaxWidth().height(52.dp),  
    shape = RoundedCornerShape( size = 14.dp),  
) {  
    Icon( imageVector = Icons.Filled.Call, contentDescription = null)  
    Spacer( Modifier.width(8.dp))  
    Text("Appeler le producteur")  
}
```

Figure 8 – Intent implicite pour appeler le producteur

3.4 Jetpack Compose (état)

Utilisation de remember et mutableStateOf pour porter l'état local (éphémère) des six champs du formulaire de publication.

```
fun EcranPublier(  
    viewModel: AnnonceViewModel,  
    onTermine: () -> Unit,  
) {  
    val profilNom by viewModel.profilNom.collectAsState()  
    val profilTelephone by viewModel.profilTelephone.collectAsState()  
  
    var nomProduit by remember { mutableStateOf( value = "") }  
    var village by remember { mutableStateOf( value = "") }  
    var quantite by remember { mutableStateOf( value = "") }  
    var prix by remember { mutableStateOf( value = "") }  
    var nomProducteur by remember { mutableStateOf( value = profilNom) }  
    var telephone by remember { mutableStateOf( value = profilTelephone) }  
    val context = LocalContext.current  
  
    Scaffold(  

```

Figure 9 – État local du formulaire Publier

3.5 Navigation (NavHost et routes)

Utilisation de NavHost, rememberNavController et de routes (dont detail/{id}) pour naviguer entre les quatre écrans, désormais avec l'onglet Mes publications en plus.

```
NavHost(navController = navController, startDestination = "accueil") {

    composable(route = "accueil") {
        EcranListe(
            viewModel = viewModel,
            navController = navController,
            onAnnonceClick = { id -> navController.navigate(route = "detail/$id") },
            onPublierClick = { navController.navigate(route = "publier") },
        )
    }

    composable(route = "mesPublications") {
        EcranMesPublications(
            viewModel = viewModel,
            navController = navController,
            onAnnonceClick = { id -> navController.navigate(route = "detail/$id") },
            onPublierClick = { navController.navigate(route = "publier") },
        )
    }

    composable(route = "detail/{id}") { backStackEntry ->
        val id = backStackEntry.arguments?.getString(key = "id")?.toIntOrNull()
        EcranDetail(
            id = id,
            viewModel = viewModel,
            onRetour = { navController.popBackStack() },
        )
    }

    composable(route = "publier") {
        EcranPublier(
            viewModel = viewModel,
            onTermine = { navController.popBackStack() },
        )
    }
}
```

Figure 10 – Graphe de navigation — AppNavigation()

3.6 Architecture MVVM (ViewModel + StateFlow)

Utilisation d'un AndroidViewModel exposant un StateFlow, et de combine() pour fusionner plusieurs Flow Room en un seul état d'écran, recalculé automatiquement à chaque changement.

```
val uiState: StateFlow<EtatUi> =
    combine(
        ...flows = dao.annoncesActives(),
        dao.parPrixCroissant(),
        dao.villages(),
        villageFiltre,
        mode,
    ) { flux ->
        @Suppress( ...names = "UNCHECKED_CAST")
        val parDate = flux[0] as List<Annonce>
        @Suppress( ...names = "UNCHECKED_CAST")
        val parPrix = flux[1] as List<Annonce>
        @Suppress( ...names = "UNCHECKED_CAST")
        val villages = flux[2] as List<String>
        val village = flux[3] as String?
        val modeCourant = flux[4] as ModeTri

        // Choix de la liste de base
        val base = when (modeCourant) {
            ModeTri.RECENTES -> parDate
            ModeTri.PRIX_CROISSANT -> parPrix
        }
        // Filtre par village
        val liste = village?.let { v -> base.filter { it.village == v } } ?: base

        EtatUi(annonces = liste, villages = villages, villageFiltre = village, mode = modeCourant)
    }.stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed(stopTimeoutMillis = 5_000),
        initialValue = EtatUi(),
    )
}
```

Figure 11 – AnnonceViewModel — combine() de plusieurs Flow en un état unique

3.7 Room

Utilisation d'une Entity, d'un DAO exposant des requêtes en Flow, et d'une Database en singleton.

```
@Dao
interface AnnonceDao {

    /** Annonces actives */
    1 Usage 1 Implementation
    @Query( value = "SELECT * FROM annonces WHERE vendue = 0 ORDER BY dateISO DESC, id DESC")
    fun annoncesActives(): Flow<List<Annonce>>

    /** Filtres par village */
    1 Implementation
    @Query( value = "SELECT * FROM annonces WHERE vendue = 0 AND village = :village ORDER BY dateISO DESC")
    fun parVillage(village: String): Flow<List<Annonce>>

    /** Tri par prix croissant */
    1 Usage 1 Implementation
    @Query( value = "SELECT * FROM annonces WHERE vendue = 0 ORDER BY prixKg IS NULL, prixKg ASC")
    fun parPrixCroissant(): Flow<List<Annonce>>

    /** Une annonce precise */
    1 Usage 1 Implementation
    @Query( value = "SELECT * FROM annonces WHERE id = :id")
    suspend fun parId(id: Int): Annonce?

    /** Liste des villages distincts */
    1 Usage 1 Implementation
    @Query( value = "SELECT DISTINCT village FROM annonces ORDER BY village ASC")
    fun villages(): Flow<List<String>>

    /** Mes propres annonces */
    1 Usage 1 Implementation
    @Query( value = "SELECT * FROM annonces WHERE estMienne = 1 ORDER BY dateISO DESC, id DESC")
    fun mesAnnonces(): Flow<List<Annonce>>

    2 Usages 1 Implementation
    @Insert
    suspend fun inserer(annonce: Annonce): Long

    2 Usages 1 Implementation
    @Update
    suspend fun modifier(annonce: Annonce)
```

Figure 12 – Le DAO — requêtes réactives sur la table annonces

5 Usages 1 Implementation

```
@Database(entities = [Annonce::class], version = 2, exportSchema = false)
abstract class AppDatabase : RoomDatabase() {
```

1 Usage 1 Implementation

```
abstract fun annonceDao(): AnnonceDao
```

5 Usages

```
companion object {
```

3 Usages

```
@Volatile private var instance: AppDatabase? = null
```

1 Usage

```
fun obtenir(context: Context): AppDatabase =
```

```
    instance ?: synchronized( lock = this) {
```

```
        instance ?: Room.databaseBuilder(
```

```
            context.applicationContext,
```

```
            klass = AppDatabase::class.java,
```

```
            name = "tsena_malagasy.db",
```

```
        )
```

```
        .fallbackToDestructiveMigration()
```

```
        .build()
```

```
        .also { instance = it }
```

```
    }
```

```
}
```

```
}
```

Figure 13 – La Database — point d'assemblage en singleton